embedded by Davidson and Harel's algorithm (right).

As the next step, we would need the methods to compute transitions to new solutions: luckily, though, we can reuse the same methods we had defined in the last section - after all, the problem space is the same, the only thing that we need to change is the cost function because we are changing our criteria to decide what a good embedding is.

Listing 17.4 Cost Function for Davidson and Harel's algorithm

```
function cost(P, w, h, lambda1, lambda2, lambda3, lambda4,
lambda5) #1
  total ← 0
  for v in P.vertices do
                            #2
    total ← total + lambda2 * ((1/x^2) + (1/y^2) + (1/(w-x)^2) +
(1/(h-y)^2))      #3
    for u in P.vertices-{v}
do                                                  #4
      total ← total + lambda1 * distance(u,
v)^2                                  #5
  for e=(u,v) in P.edges do
                                              #6
    total ← total + lambda3 * distance(u,
v)^2                                    #7
    for z in P.vertices-{u,v}
do                                                 #8
      total ← total + lambda5 / distance(e,
z)^2                                  #9
  total ← total + lambda4 * P.intersections()
                            #10
  return total
```